

IME 775 – Lecture 11

Bayesian Tools: Bayes' Theorem, Entropy, and Cross-Entropy

1. From Distributions to Decisions

Chapter 5 gave us the language of probability: distributions, expectations, covariance. Now we need **tools** to:

- **Update beliefs** when new evidence arrives (Bayes' theorem)
- **Quantify uncertainty** in a distribution (entropy)
- **Measure how well** one distribution approximates another (cross-entropy)

These tools are foundational to loss functions, generative models, and Bayesian inference in deep learning.

2. Conditional Probability

Definition

The probability of A given that B has occurred:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

Intuition: We restrict the sample space to outcomes where B is true, then ask how likely A is within that restricted space.

Product Rule (from rearranging)

$$P(A \cap B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

This identity is the bridge to Bayes' theorem.

Workout: In a dataset of 1000 emails, 400 are spam. Among spam, 80% contain the word "free." Among non-spam, 10% contain "free." What is $P(\text{spam} | \text{"free"})$?

Solution:

- $P(\text{spam}) = 0.4, P(\text{not spam}) = 0.6$
- $P(\text{"free"} | \text{spam}) = 0.8$
- $P(\text{"free"} | \text{not spam}) = 0.1$
- $P(\text{"free"}) = 0.4 \times 0.8 + 0.6 \times 0.1 = 0.32 + 0.06 = 0.38$
- $P(\text{spam} | \text{"free"}) = \frac{0.4 \times 0.8}{0.38} = \frac{0.32}{0.38} \approx 0.842$

Seeing "free" raises the spam probability from 40% to 84%.

3. Bayes' Theorem

The Formula

$$P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta) \cdot P(\theta)}{P(\mathcal{D})}$$

Terminology

Term	Name	Meaning
$P(\theta \mathcal{D})$	Posterior	Updated belief about θ after seeing data
$P(\mathcal{D} \theta)$	Likelihood	How probable the data is under

		parameter θ
$P(\theta)$	Prior	Initial belief about θ before seeing data
$P(\mathcal{D})$	Evidence (marginal likelihood)	Total probability of the data under all possible θ

The Evidence Term

$$P(\mathcal{D}) = \int P(\mathcal{D} | \theta) P(\theta) d\theta \quad (\text{continuous})$$

$$P(\mathcal{D}) = \sum_{\theta} P(\mathcal{D} | \theta) P(\theta) \quad (\text{discrete})$$

This is a normalizing constant that ensures the posterior sums/integrates to 1. It is often intractable for complex models, which motivates approximate methods (variational inference, MCMC).

Bayes' Theorem as a Learning Rule

$$\underbrace{P(\theta | \mathcal{D})}_{\text{what I believe now}} = \frac{\overbrace{P(\mathcal{D} | \theta)}^{\text{what data tells me}} \cdot \overbrace{P(\theta)}^{\text{what I believed before}}}{\underbrace{P(\mathcal{D})}_{\text{normalization}}}$$

Key insight: Bayes' theorem is the mathematically optimal way to update beliefs given new evidence. It balances prior knowledge with observed data.

Deeper explanation:

Why is this "optimal"? Under the axioms of probability, there is no other consistent way to update beliefs. Cox's theorem (1946) proves that any system of belief updating that satisfies basic logical consistency constraints (e.g., if you believe A makes B more likely, you cannot also believe A makes B less likely) must reduce to Bayes' theorem. It is not a modeling choice — it is the unique consequence of requiring logical coherence.

In practice: with little data, the posterior is dominated by the prior (your assumptions matter). With abundant data, the likelihood dominates and the prior is "washed out" (the data speaks for itself). The transition between these regimes is smooth and automatic.

Workout: A disease affects 1% of the population. A test has 95% sensitivity (true positive rate) and 90% specificity (true negative rate). If a person tests positive, what is the probability they have the disease?

Solution:

- $P(\text{disease}) = 0.01, P(\text{no disease}) = 0.99$
- $P(+ \mid \text{disease}) = 0.95, P(+ \mid \text{no disease}) = 0.10$
- $P(+) = 0.01 \times 0.95 + 0.99 \times 0.10 = 0.0095 + 0.099 = 0.1085$
- $P(\text{disease} \mid +) = \frac{0.0095}{0.1085} \approx 0.088$

Despite 95% sensitivity, a positive test only gives ~8.8% disease probability. The low base rate (1% prevalence) makes false positives dominate.

ML lesson: This is why class-imbalanced datasets are problematic — a classifier can achieve high accuracy by predicting the majority class, but performs poorly on the rare class.

4. Bayes' Theorem in Machine Learning

Classification

For a classifier with classes C_k and feature vector $\vec{x}$:

$$P(C_k \mid \vec{x}) = \frac{P(\vec{x} \mid C_k) \cdot P(C_k)}{P(\vec{x})}$$

- $P(\vec{x} \mid C_k)$: class-conditional density (how data looks in each class)

- $P(C_k)$: class prior (how common each class is)
- Decision rule: pick $\hat{k} = \arg \max_k P(C_k | \vec{x})$

Since $P(\vec{x})$ is constant across classes, we only need:

$$\hat{k} = \arg \max_k P(\vec{x} | C_k) \cdot P(C_k)$$

Parameter Estimation

For model parameters $\vec{w}$ given training data $\mathcal{D}$:

$$P(\vec{w} | \mathcal{D}) \propto P(\mathcal{D} | \vec{w}) \cdot P(\vec{w})$$

Approach	Prior $P(\vec{w})$	Finds
MLE	Ignored (uniform)	$\hat{\vec{w}} = \arg \max_{\vec{w}} P(\mathcal{D} \vec{w})$
MAP	Specified	$\hat{\vec{w}} = \arg \max_{\vec{w}} P(\mathcal{D} \vec{w}) P(\vec{w})$
Full Bayesian	Specified	Entire posterior $P(\vec{w} \mathcal{D})$

Key distinction: Both MLE and MAP return a **single point estimate** — one specific value of $\vec{w}$. Only full Bayesian inference returns the entire distribution over $\vec{w}$, quantifying how uncertain you are about the parameters.

Method	What it does	Output
MLE	Finds parameters that make observed data most probable, ignoring any prior beliefs	Single point estimate of $\vec{w}$
MAP	Same as MLE but incorporates prior beliefs about likely parameter values; equivalent to adding regularization	Single point estimate of $\vec{w}$
Full Bayesian	Computes the complete posterior distribution $P(\vec{w} \mathcal{D})$, preserving all uncertainty information	Distribution over $\vec{w}$

As data grows large, MLE and MAP converge to the same answer (the prior becomes negligible), and the full Bayesian posterior concentrates into a narrow peak around that answer.

We will cover MLE and MAP in detail in Lecture 12.

5. Information Theory: Entropy

Motivation: Quantifying Surprise

How "surprised" are you when event x with probability $p(x)$ occurs?

$$I(x) = -\log_2 p(x) \quad (\text{in bits})$$

- $p(x) = 1$ (certain): $I(x) = 0$ bits (no surprise)
- $p(x) = 0.5$: $I(x) = 1$ bit
- $p(x) = 0.01$ (rare): $I(x) \approx 6.64$ bits (very surprising)

Intuition: Rare events carry more information when they occur. A weather report saying "sunny" in Phoenix carries less information than "sunny" in Seattle in November.

Shannon Entropy

The **expected surprise** (average information content) of a distribution:

$$H(X) = -\sum_i p(x_i) \log_2 p(x_i) = E[-\log_2 p(X)]$$

For continuous distributions (differential entropy):

$$H(X) = -\int p(x) \log p(x) dx$$

Bits vs Nats

The unit of entropy depends on the logarithm base:

Base	Unit	Convention	$H(\text{fair coin})$
$\log_2$	bits	Information theory, compression	1 bit
$\ln$ (base e)	nats	ML, statistics, calculus	0.693 nats

Conversion: $1 \text{ nat} = 1 / \ln 2 \approx 1.443$ bits, or equivalently $1 \text{ bit} = \ln 2 \approx 0.693$ nats.

Why two conventions? Shannon's original formulation uses $\log_2$ because bits map directly to binary digits — the physical unit of digital storage. Machine learning uses $\ln$ because calculus is cleaner: $\frac{d}{dx} \ln x = 1/x$ with no extra constants, so gradients of log-likelihoods are simpler. PyTorch's `cross_entropy`, `kl_div`, and `NLLLoss` all use $\ln$ internally, so their outputs are in nats.

For optimization it does not matter: switching bases multiplies the loss by a constant ($\ln 2$), which does not change where the minimum is. So MLE, MAP, and gradient descent produce the same parameters regardless of which base you use.

In these notes, we use $\log_2$ (bits) for discrete examples where the "number of yes/no questions" interpretation is intuitive, and $\ln$ (nats) for continuous/Gaussian formulas and ML derivations where calculus is involved. Any formula written with bare "log" (no subscript) uses $\ln$ by convention in the ML context.

Properties of Entropy

- $H(X) \geq 0$ for discrete distributions
- $H(X) = 0$ iff X is deterministic (one outcome has $p = 1$)
- $H(X)$ is maximized when all outcomes are equally likely (uniform distribution)
- For K equally likely outcomes: $H(X) = \log_2 K$

Entropy Examples

Distribution	$H(X)$	Interpretation
Fair coin: (0.5, 0.5)	1 bit	Maximum uncertainty for binary

Biased coin: (0.9, 0.1)	0.47 bits	More predictable
Certain: (1, 0)	0 bits	No uncertainty
Fair die: uniform over 6	2.58 bits	$\log_2 6$
Uniform over 256	8 bits	$\log_2 256$ (1 byte)

Why bits? Using $\log_2$, entropy measures the average number of yes/no questions needed to identify the outcome. A fair coin needs exactly 1 question. A fair die needs ~ 2.58 questions on average (binary search).

Deeper explanation:

Shannon entropy has an operational meaning tied to data compression. The source coding theorem (Shannon, 1948) proves that the minimum average number of bits per symbol needed to losslessly encode messages from source X is exactly $H(X)$. You cannot compress below entropy without losing information.

This means entropy is not just a mathematical abstraction — it is the physical limit of how compactly information can be stored. A source with $H = 2$ bits/symbol requires at least 2 bits per symbol on average, no matter how clever the encoding.

Workout: Compute $H(X)$ for $p = (0.25, 0.25, 0.25, 0.25)$ and $p = (0.7, 0.1, 0.1, 0.1)$.

Solution:

Uniform: $H = -4 \times 0.25 \log_2 0.25 = -4 \times 0.25 \times (-2) = 2$ bits

Non-uniform:

$$\begin{aligned}
 H &= -(0.7 \log_2 0.7 + 3 \times 0.1 \log_2 0.1) \\
 &= -(0.7 \times (-0.515) + 0.3 \times (-3.322)) \\
 &= -(-0.360 - 0.997) = 1.357 \text{ bits}
 \end{aligned}$$

The non-uniform distribution has lower entropy (less uncertainty), as expected.

Differential Entropy: Continuous Case

For continuous distributions, the entropy formula uses an integral instead of a sum and is called **differential entropy**. Unlike discrete entropy, differential entropy can be negative.

Gaussian differential entropy:

For $X \sim \mathcal{N}(\mu, \sigma^2)$:

$$H(X) = \frac{1}{2} \ln(2\pi e \sigma^2) = \frac{1}{2} (1 + \ln(2\pi \sigma^2))$$

Key observations:

- Entropy depends only on σ , not on μ – shifting the distribution does not change uncertainty
- Larger $\sigma \rightarrow$ higher entropy (more spread = more uncertainty)
- This is the **maximum entropy** among all continuous distributions with the same variance (recall from Lecture 10)

Derivation sketch: Substitute $p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$ into $H = -\int p(x) \ln p(x) dx$. The $\ln p(x)$ expands into two terms: a constant $-\frac{1}{2} \ln(2\pi\sigma^2)$ and a quadratic $-\frac{(x-\mu)^2}{2\sigma^2}$. Taking expectations, the second term becomes $-\frac{1}{2}$ (since $E[(X - \mu)^2] = \sigma^2$). Adding gives $H = \frac{1}{2} \ln(2\pi\sigma^2) + \frac{1}{2} = \frac{1}{2} \ln(2\pi e \sigma^2)$.

Workout: Compute and compare the differential entropy for $\sigma = 1$ and $\sigma = 3$.

Solution:

- $\sigma = 1: H = \frac{1}{2} \ln(2\pi e \cdot 1) = \frac{1}{2} \ln(17.08) \approx 1.42$ nats
- $\sigma = 3: H = \frac{1}{2} \ln(2\pi e \cdot 9) = \frac{1}{2} \ln(153.7) \approx 2.52$ nats

Tripling the standard deviation adds about 1.1 nats of entropy. In general, multiplying σ by k adds $\ln k$ nats, since $\frac{1}{2} \ln(2\pi e k^2 \sigma^2) = \frac{1}{2} \ln(2\pi e \sigma^2) + \ln k$.

Multivariate Gaussian:

For $\vec{X} \sim \mathcal{N}(\vec{\mu}, \Sigma)$ in d dimensions:

$$H(\vec{X}) = \frac{d}{2} \ln(2\pi e) + \frac{1}{2} \ln |\Sigma|$$

where $|\Sigma|$ is the determinant of the covariance matrix. The determinant captures the "volume" of the uncertainty ellipsoid — larger determinant means more spread in the joint distribution.

6. Cross-Entropy

Definition

The cross-entropy between the **true** distribution p and a **model** distribution q :

$$H(p, q) = - \sum_i p(x_i) \log q(x_i) = E_p[-\log q(X)]$$

Interpretation

Cross-entropy measures the **average number of bits needed** to encode outcomes from p using an encoding optimized for q .

- If $q = p$: $H(p, q) = H(p)$ (optimal encoding)
- If $q \neq p$: $H(p, q) > H(p)$ (suboptimal — extra bits wasted)

The **extra cost** of using q instead of p is $H(p, q) - H(p) = D_{KL}(p||q)$ (KL divergence — Lecture 12).

Cross-Entropy as a Loss Function

In classification, the true label is a one-hot distribution $\vec{y}$ and the model outputs probabilities $\hat{\vec{y}}$:

$$\mathcal{L} = H(\vec{y}, \hat{\vec{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Binary case ($K = 2$, logistic regression):

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Why cross-entropy and not MSE for classification?

The gradient of cross-entropy with respect to the logits is $\hat{y} - y$ — clean and bounded. MSE produces gradients that vanish when the sigmoid saturates (near 0 or 1), causing extremely slow learning. Cross-entropy penalizes confident wrong predictions severely (log goes to $-\infty$), giving strong gradients exactly when the model needs correction most.

Workout: True label: cat (one-hot: $[1, 0, 0]$). Model predicts $\hat{y} = [0.7, 0.2, 0.1]$. Compute cross-entropy loss.

Solution:

$$\mathcal{L} = -(1 \cdot \log 0.7 + 0 \cdot \log 0.2 + 0 \cdot \log 0.1) = -\log 0.7 \approx 0.357$$

If the model had predicted $[0.1, 0.7, 0.2]$ (wrong class most likely):

$$\mathcal{L} = -\log 0.1 \approx 2.303$$

The loss is much higher for the confident wrong prediction, as desired.

Continuous Cross-Entropy: Gaussian Case

For two Gaussians $p = \mathcal{N}(\mu_1, \sigma_1^2)$ and $q = \mathcal{N}(\mu_2, \sigma_2^2)$:

$$H(p, q) = \frac{1}{2} \ln(2\pi\sigma_2^2) + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2}$$

Derivation: $H(p, q) = -E_p[\ln q(X)]$. Since $\ln q(x) = -\frac{1}{2} \ln(2\pi\sigma_2^2) - \frac{(x-\mu_2)^2}{2\sigma_2^2}$, the expectation under p gives $E_p[(X - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$ (bias-variance decomposition), yielding the formula above.

Special case – when $p = q$: Cross-entropy reduces to the entropy of p :

$$H(p, p) = \frac{1}{2} \ln(2\pi\sigma_1^2) + \frac{1}{2} = \frac{1}{2} \ln(2\pi e\sigma_1^2) = H(p)$$

Workout: Compute $H(p, q)$ for $p = \mathcal{N}(0, 1)$ and $q = \mathcal{N}(1, 4)$.

Solution:

$$\begin{aligned} H(p, q) &= \frac{1}{2} \ln(2\pi \cdot 4) + \frac{1 + (0 - 1)^2}{2 \cdot 4} = \frac{1}{2} \ln(25.13) + \frac{2}{8} \\ &= 1.61 + 0.25 = 1.86 \text{ nats} \end{aligned}$$

Compare with the entropy of p : $H(p) = \frac{1}{2} \ln(2\pi e) \approx 1.42$ nats.

The difference $H(p, q) - H(p) = 1.86 - 1.42 = 0.44$ nats is the KL divergence $D_{KL}(p||q)$, which measures the cost of the mismatch between the two Gaussians.

ML connection – regression (why MSE is cross-entropy in disguise):

In linear regression we assume the data follows:

$$y = \vec{w}^T \vec{x} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Why does $\vec{w}^T \vec{x}$ become the mean of a Gaussian?

$\vec{w}^T \vec{x}$ is not itself random or Gaussian — it is a deterministic number for any given input $\vec{x}$ and fixed weights $\vec{w}$. The Gaussian assumption is placed only on the **noise** ϵ . The key step is a basic property of Gaussians: adding a constant c to a Gaussian variable shifts its mean by c and leaves the variance unchanged. If $\epsilon \sim \mathcal{N}(0, \sigma^2)$, then:

$$y = \underbrace{\vec{w}^T \vec{x}}_{\text{constant}} + \underbrace{\epsilon}_{\sim \mathcal{N}(0, \sigma^2)} \implies y \sim \mathcal{N}(\vec{w}^T \vec{x}, \sigma^2)$$

So the model says: "For input $\vec{x}$, I predict the output is **centered** at $\vec{w}^T \vec{x}$, with residual uncertainty σ^2 around that center."

Why is zero-mean Gaussian noise reasonable? In many physical systems, measurement errors arise from the sum of many small independent sources (sensor imprecision, environmental fluctuations, rounding). The Central Limit Theorem guarantees such sums converge to a Gaussian. The zero-mean assumption says these errors are unbiased — they do not systematically push observations above or below the true value $\vec{w}^T \vec{x}$.

This means each observation y_i given input $\vec{x}_i$ is drawn from a Gaussian:

$$p(y_i | \vec{x}_i, \vec{w}) = \mathcal{N}(y_i | \vec{w}^T \vec{x}_i, \sigma^2) = \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{(y_i - \vec{w}^T \vec{x}_i)^2}{2\sigma^2} \right)$$

The negative log-likelihood for N data points is:

$$-\ell(\vec{w}) = -\sum_{i=1}^N \log p(y_i | \vec{x}_i, \vec{w}) = \frac{N}{2} \ln(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \vec{w}^T \vec{x}_i)^2$$

The first term is a constant (does not depend on $\vec{w}$). Minimizing over $\vec{w}$ gives:

$$\hat{\vec{w}}_{MLE} = \arg \min_{\vec{w}} \sum_{i=1}^N (y_i - \vec{w}^T \vec{x}_i)^2 = \arg \min_{\vec{w}} \text{MSE}$$

The punchline: MSE loss is not an arbitrary design choice — it is the exact negative log-likelihood (i.e., cross-entropy) when you assume Gaussian noise. The σ^2 in the denominator acts as a scaling factor: larger assumed noise $\rightarrow$ smaller gradients $\rightarrow$ slower learning, which makes physical sense (if you expect high noise, each residual carries less signal).

This also explains why MSE is appropriate for regression but not for classification: classification outputs are categorical (Bernoulli/Categorical), not Gaussian. Using cross-entropy for classification and MSE for regression is the same principle — maximum likelihood — applied to different distributional assumptions.

Task	Assumed distribution	Negative log-likelihood	Loss function
Regression	Gaussian $\mathcal{N}(\hat{y}, \sigma^2)$	$\frac{1}{2\sigma^2}(y - \hat{y})^2 + \text{const}$	MSE
Binary classification	Bernoulli $\text{Ber}(\hat{y})$	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	Binary cross-entropy
Multi-class	Categorical $\text{Cat}(\hat{y})$	$-\sum_k y_k \log \hat{y}_k$	Cross-entropy

Relationship Summary

$$\underbrace{H(p, q)}_{\text{cross-entropy}} = \underbrace{H(p)}_{\text{entropy}} + \underbrace{D_{KL}(p||q)}_{\text{KL divergence}}$$

Since $H(p)$ is constant with respect to model parameters, minimizing cross-entropy is equivalent to minimizing KL divergence from p to q .

7. Connection to Chapter 5

Chapter 5 Concept	Chapter 6 Extension
$P(A, B) = P(A B)P(B)$ (product rule)	Bayes' theorem (invert the conditioning)
Gaussian $\mathcal{N}(\mu, \sigma^2)$	Entropy of Gaussian: $H = \frac{1}{2} \ln(2\pi e \sigma^2)$
Categorical $\text{Cat}(\vec{\theta})$	Cross-entropy loss for multi-class

Bernoulli $\text{Ber}(\theta)$

Binary cross-entropy loss

Expected value $E[X]$

Entropy is $E[-\log p(X)]$

Key Takeaways

1. **Conditional probability** restricts the sample space; the product rule connects joint and conditional
 2. **Bayes' theorem** is the unique logically consistent rule for updating beliefs given evidence
 3. **Entropy** quantifies uncertainty — it equals the theoretical minimum bits for encoding
 4. **Cross-entropy** measures encoding cost under a wrong model — it is the standard classification loss
 5. Minimizing cross-entropy = minimizing KL divergence from true to predicted distribution (Lecture 12)
-

PyTorch Connection

```
import torch
import torch.nn.functional as F

# Cross-entropy loss (multi-class)
logits = torch.tensor([[2.0, 1.0, 0.5]]) # raw model output
target = torch.tensor([0]) # true class index
loss = F.cross_entropy(logits, target)
print(f"Cross-entropy loss: {loss.item():.4f}") # ≈ 0.4076

# Binary cross-entropy
pred = torch.tensor([0.7]) # predicted probability
label = torch.tensor([1.0]) # true label
bce = F.binary_cross_entropy(pred, label)
```

```

print(f"Binary CE: {bce.item():.4f}") # ≈ 0.3567

# Entropy of a distribution
probs = torch.tensor([0.25, 0.25, 0.25, 0.25])
entropy = -torch.sum(probs * torch.log2(probs))
print(f"Entropy (uniform 4): {entropy.item():.4f} bits") # = 2.0

probs2 = torch.tensor([0.7, 0.1, 0.1, 0.1])
entropy2 = -torch.sum(probs2 * torch.log2(probs2))
print(f"Entropy (skewed): {entropy2.item():.4f} bits") # ≈ 1.357

# Bayes' theorem in code
p_spam = 0.4
p_free_given_spam = 0.8
p_free_given_not_spam = 0.1
p_free = p_spam * p_free_given_spam + (1 - p_spam) *
p_free_given_not_spam
p_spam_given_free = (p_free_given_spam * p_spam) / p_free
print(f"P(spam | 'free') = {p_spam_given_free:.4f}") # ≈ 0.842

```